

ATLAS NEXUS

Resumen completo del proyecto

VERSIÓN DETALLADA

Este documento explica **absolutamente todo** lo que está hecho en el proyecto Atlas Nexus, primero en palabras simples para cualquier persona del equipo o stakeholder, y después con el detalle técnico de cada tecnología, módulo y decisión de diseño.

Atlas Nexus es un acelerador inteligente para negocios emergentes que ya usan **Clover POS**. Tomamos sus ventas, sus productos, sus clientes recurrentes y los cruzamos con inteligencia artificial y machine learning para dar al dueño del negocio respuestas que hoy no tiene: qué vendió, por qué, qué se viene la semana que entra, qué cliente está por irse y qué producto va a quedar sin stock.

Fecha: 23 de mayo de 2026

Repositorio: track_integracionpagos · Branch: main

Estado: MVP single-tenant funcional (Web + Backend + ML + IA + Android parcial)

Contenido

1. Qué es Atlas Nexus (en una frase)
2. Para quién está pensado
3. Qué se hizo, explicado en palabras simples
 - 3.1. Registro e inicio de sesión
 - 3.2. Dashboard principal del negocio
 - 3.3. Productos y categorías
 - 3.4. Sincronización con Clover (la caja registradora)
 - 3.5. Insights con Inteligencia Artificial
 - 3.6. Forecasting / Predicción de demanda y stock
 - 3.7. Patrones de compra
 - 3.8. Meta mensual y configuración
 - 3.9. App móvil Android
 - 3.10. Datos de demostración
 - 3.11. Ventas en tiempo real
4. Cómo se ve el flujo end-to-end
5. Tecnologías usadas — vista panorámica
6. Backend en profundidad (FastAPI + SQLAlchemy + Postgres)
7. Modelo de datos (tablas y relaciones)
8. Endpoints / API REST disponibles
9. Frontend en profundidad (React + Vite + Tailwind + Recharts)
10. Inteligencia Artificial conversacional (Groq + Llama 3.3)
11. Machine Learning de forecasting (XGBoost + pandas)
12. Integración con Clover POS
13. Streaming en tiempo real (SSE)
14. Autenticación y seguridad
15. Infraestructura local y deploy (Docker Compose)
16. App Android (Kotlin + Gradle) — estado actual
17. Estado del proyecto, deuda técnica y próximos pasos
18. Roadmap por fases
19. Modelo de negocio sugerido

Lo que se hizo, en palabras simples

1. Qué es Atlas Nexus (en una frase)

Es una aplicación web (con app móvil Android en camino) que se conecta a una caja registradora Clover y le da al dueño del negocio un panel con información clara: cuánto vendió hoy, qué le va a faltar la semana que viene, qué clientes están dejando de comprarle, y un asistente con inteligencia artificial al que puede preguntarle cosas en español como si fuera un asesor.

2. Para quién está pensado

Para comerciantes chicos y medianos — una cafetería, un kiosco, una panadería, una tienda de barrio, un local de ropa — que ya usan Clover como sistema de cobro y quieren entender mejor su negocio sin tener que ser contadores ni saber Excel. La aplicación habla en castellano, es directa, y no asume conocimientos técnicos.

3. Qué se hizo, explicado en palabras simples

3.1. Registro e inicio de sesión

El comerciante entra al sitio, crea una cuenta con su email, el nombre de su tienda y una contraseña. La próxima vez vuelve a entrar con email y contraseña y queda dentro de su panel. Cada cuenta es independiente: las ventas y los datos de un negocio no se mezclan con los de otro. La contraseña no se guarda como la escribió: se guarda *hasheada* (es decir, transformada de manera que ni siquiera nosotros podemos verla).

3.2. Dashboard principal del negocio

Apenas entra, el comerciante ve un tablero con los números importantes del negocio. Puede elegir si quiere mirar **hoy, esta semana, este mes o este año**, y todo el tablero se recalcula al instante.

Tarjetas principales (KPIs) que se muestran:

- **Ingresos totales** del período, con flecha arriba o abajo comparando contra el período anterior.
- **Cantidad de ventas** hechas en el período.
- **Ticket promedio**: cuánto gasta en promedio cada cliente por compra.
- **Cantidad de productos** que tiene cargados.
- **Cantidad de categorías** en que tiene organizados sus productos.
- **Cientes únicos** identificados en el período.
- **Tasa de retorno**: qué porcentaje de clientes volvió a comprar.

Gráficos que se muestran:

- **Línea de ventas a lo largo del tiempo** — un día por punto, para ver cuándo vendió más y cuándo menos.
- **Distribución por método de pago** (Tarjeta vs QR) en un gráfico de torta.
- **Distribución por marca de tarjeta** (Visa, Mastercard, Amex).
- **Top 10 productos** por ingresos, con cantidad de unidades vendidas.
- **Categorías con más ingresos**.
- **Top 5 clientes** por gasto total, con la cantidad de compras que hicieron.

Otras cosas que aparecen en la página principal:

- Un **briefing del día** generado con IA: 2 o 3 oraciones que combinan un dato concreto, una oportunidad o riesgo, y una acción específica que el dueño puede tomar HOY.
- **Alertas inteligentes:** clientes en riesgo de irse, días en que se vendió por encima del promedio, productos que dejaron de venderse, avances hacia la meta del mes.
- Un botón de **Sync Clover** que va y trae las últimas ventas de la caja registradora con un click.
- Botón de **refresh** para actualizar los datos manualmente, y selector de período.

3.3. Productos y categorías

El comerciante puede dar de alta los productos que vende, ponerles un precio y agruparlos en categorías (por ejemplo: Bebidas, Comidas, Postres). Cada producto puede pertenecer a varias categorías a la vez. Esto sirve para que los reportes después salgan ordenados y la IA pueda razonar sobre 'qué categoría te está rindiendo más' en vez de hablar producto por producto.

3.4. Sincronización con Clover (la caja registradora)

Si el negocio ya tiene Clover configurado (token de acceso del comercio), el sistema puede traer las ventas reales de Clover y meterlas en Atlas. Hay dos formas de hacerlo:

- **Sync manual:** el comerciante aprieta un botón y el backend va a la API de Clover, trae las últimas órdenes, las convierte en ventas de Atlas y las guarda. Si una venta ya estaba importada, no la duplica.
- **Webhook:** cuando ocurre una venta en Clover, Clover llama a nuestro endpoint `/clover/webhook` avisando del evento, y Atlas va a buscar el detalle de esa orden y la guarda. Esto permite tener el panel casi en vivo.

Las ventas importadas se mapean a la misma estructura que las ventas creadas en Atlas: número de factura (con prefijo CLV- para identificar que vino de Clover), método de pago, tipo y marca de tarjeta, total, fecha y línea por línea de productos. Si un producto que viene de Clover no existe en Atlas, se crea automáticamente.

3.5. Insights con Inteligencia Artificial

Esta es una de las funciones diferenciadoras. Está dividida en tres partes:

a) Briefing del día con IA

Cada vez que el comerciante entra a la app, se genera un mini-texto de 2 o 3 oraciones que le resume el día y le sugiere una acción concreta. Ejemplo de lo que devolvería: *"Hoy llevás \$42.300 con 18 ventas. Tu cliente más fiel no compra hace 12 días: mandale un saludo o un combo personalizado. Probá ofrecer el combo café+medialuna entre las 8 y 10am para subir el ticket promedio."* El texto se genera con un modelo de lenguaje (Llama 3.3 corriendo en Groq) al que se le pasan los datos del negocio del día y la meta mensual.

b) Chat con la IA del negocio

El comerciante puede hacerle preguntas en lenguaje natural como *"¿qué producto me rindió más esta semana?"*, *"¿cuánto le vendí a Pedro el mes pasado?"*, *"¿estoy mejor que el mes anterior?"*. La IA recibe un contexto con los datos reales del período elegido (ingresos, ventas, ticket promedio, top productos, clientes únicos, retorno) y responde en castellano, breve y accionable. Mantiene el historial de la conversación así puede responder repreguntas (*"¿y al revés?"*, *"¿en qué horario?"*).

c) Alertas automáticas

Sin que el comerciante pregunte, el sistema le muestra alertas cuando detecta cuatro cosas distintas:

- **Clientes en riesgo:** gente que compraba seguido y hace 14 días o más que no aparece. Sugiere mandarles una promoción.
- **Día por encima del promedio:** cuando el día actual va 20% o más arriba que el promedio diario de los últimos 7 días, lo celebra con un mensaje verde.
- **Productos sin movimiento:** productos que se vendieron alguna vez en los últimos 30 días pero no se vendieron en los últimos 7. Nombra los dos primeros y dice cuántos más hay así.
- **Avance de meta mensual:** si la meta mensual está fijada, avisa cuando se llegó al 75% y cuando se superó el 100%.

3.6. Forecasting / Predicción de demanda y stock

Es la parte de Machine Learning. El sistema mira el histórico de ventas (agrupado por semana, por producto) y entrena un modelo que predice cuánto se va a vender de cada producto la semana que viene. Cruza esa predicción con el stock actual y devuelve una **lista de compras recomendada**: cuántas unidades de cada producto le conviene pedir.

Para cada producto el sistema muestra:

- Stock actual.
- Demanda predicha para los próximos 7 días.
- Compra recomendada (predicción menos stock actual, nunca negativa).
- Confianza del modelo (0 a 1), basada en qué tan bien el modelo predijo el histórico.
- Tipo de modelo usado: *xgboost* si hay al menos 4 semanas de datos, o *promedio móvil* de las últimas 4 semanas como fallback cuando hay poca data.
- Nivel de alerta: **crítico** (necesita comprar mucho), **moderado** o **ok**.

El comerciante también puede ir a una pantalla de **stock** y poner a mano cuántas unidades tiene de cada producto, para que la recomendación sea más afilada.

3.7. Patrones de compra

Hay una pantalla dedicada a entender qué pasa con los clientes: cuándo compran, qué métodos de pago prefieren, y cómo evoluciona el comportamiento de compra a lo largo del tiempo. Es la base sobre la que más adelante se va a montar el Market Basket Analysis (qué productos se venden juntos) y la segmentación RFM (clientes VIP, frecuentes, dormidos, nuevos).

3.8. Meta mensual y configuración

En la pantalla de **Configuración** el dueño puede fijar una meta de facturación mensual. A partir de ahí toda la app la usa: el briefing la incluye en el contexto, las alertas avisan cuando se pasa el 75% y el 100%, y el dashboard muestra el avance.

3.9. App móvil Android (estado actual: parcial)

Existe una app nativa Android escrita en Kotlin, en la carpeta *CloverIAMarketing/*, con su propia arquitectura documentada. Por ahora tiene **pantalla de login y un dashboard básico**. Falta construir el resto: productos, categorías, ventas, insights, forecasting, settings, notificaciones push y modo offline. Está planificado como Fase 1 del roadmap.

3.10. Datos de demostración

Para que cualquier persona pueda probar el sistema sin tener Clover ni ventas reales, viene un script *seed_demo.py* que carga datos de un negocio ficticio: una cafetería con 4 categorías, 22 productos, ~1.100 ventas distribuidas en los últimos 45 días, niveles de stock por producto, y emails recurrentes para poder ver las métricas de retención. El login de demo es *pedro@demo.com / demo123*.

3.11. Ventas en tiempo real (streaming)

Cuando se crea una venta nueva (manual, importada de Clover o vía webhook), el backend la *broadcastea* por un canal en vivo (Server-Sent Events). Cualquier cliente conectado al canal recibe el evento sin tener que recargar la página. Es la base para que en una pantalla de caja se vean entrar las ventas como un feed.

4. Cómo se ve el flujo end-to-end

Para que quede claro cómo se conectan todas las piezas, este es el recorrido típico de un dato desde que ocurre la venta hasta que la ve el dueño en su panel:

- **1.** El cliente paga en el negocio con la terminal Clover.
- **2.** Clover registra la venta y dispara un webhook a nuestra API o nosotros corremos el sync manual.
- **3.** El backend recibe el evento, va a la API REST de Clover y trae la orden completa con sus line items y pagos.
- **4.** El servicio *clover_service* mapea la orden Clover a una Sale + SaleItems de Atlas, creando productos nuevos si hace falta.
- **5.** La venta se guarda en Postgres y se emite un evento por SSE a los clientes conectados.
- **6.** El dashboard, al refrescar o al recibir el evento, llama a */dashboard/summary* que recalcula KPIs, top productos, top clientes, métodos de pago, etc.
- **7.** Las pantallas de Insights llaman a */insights/briefing* y */insights/alerts*; el briefing arma un prompt con los datos y se lo pasa a Llama 3.3 vía Groq.
- **8.** Cuando el dueño abre Forecasting, el endpoint */forecasting/recommendations* agrega ventas por semana, arma features, entrena XGBoost y devuelve la lista de compras recomendada.

Tecnologías y arquitectura, en detalle

5. Tecnologías usadas — vista panorámica

La aplicación es una arquitectura clásica de 3 capas (frontend SPA + backend API + base relacional) más capas opcionales de IA, ML y streaming. Todo corre en contenedores Docker para que el dev local sea *git clone && docker compose up*.

Capa	Stack
Frontend	React 18, React Router 6, Vite 5, Tailwind CSS 3, Recharts 2, Lucide icons
Backend	Python 3, FastAPI 0.115, Uvicorn, Pydantic Settings, SQLAlchemy 2.0
Base de datos	PostgreSQL 15 (Alpine), tipos UUID, ENUMs nativos, índices únicos
Autenticación	JWT con python-jose (HS256), passlib + bcrypt para hash
IA conversacional	Groq SDK + modelo Llama 3.3 70B versátil
Machine Learning	XGBoost 2.1, scikit-learn 1.5, pandas 2.2, numpy 1.26
Streaming	Server-Sent Events vía sse-starlette
Integración POS	Cliente HTTP a Clover REST API (httpx 0.28)
DevOps	Docker, docker-compose, Dockerfiles por servicio, volúmenes para hot-reload
Mobile	Android nativo en Kotlin con Gradle KTS (CloverIAMarketing/)
Migraciones	Alembic instalado (aún no en uso — DDL al startup por ahora)
Testing	pytest 8.3 (estructura instalada, suite por crecer)

6. Backend en profundidad

El backend es una API REST monolítica escrita con FastAPI. La aplicación se inicializa en *app/main.py* usando un *lifespan* async: al arrancar, se crea el engine de SQLAlchemy y se materializan las tablas con *init_db*; al apagar, se limpian conexiones. Se registran 9 routers con responsabilidades bien separadas:

Router	Responsabilidad
auth.py	Registro, login, refresh de tokens, lectura del usuario actual, edición de meta mensual.
sales.py	Crear ventas manualmente, listar con filtros (método de pago, marca de tarjeta), paginación skip/limit
products.py	CRUD de productos, asignación de categorías (many-to-many).
categories.py	CRUD de categorías por usuario.
dashboard.py	Endpoint maestro /dashboard/summary que devuelve todos los KPIs, gráficos, top productos, top cliente
insights.py	Briefing diario con IA, chat conversacional con historial, alertas automáticas (en-riesgo/milestone/slow)

forecasting.py	Genera recomendaciones de compra entrenando XGBoost, lista y actualiza stock por producto.
clover.py	Recibe webhooks de Clover y trae órdenes con sync manual.
stream.py	Server-Sent Events para empujar ventas nuevas en tiempo real.

El módulo *app/services/* contiene la lógica que no es HTTP: *auth.py* con hash y JWT, *clover_service.py* con el cliente HTTP a Clover y el mapeo orden→Sale, *ml_forecasting.py* con todo el pipeline de ML, y *event_manager.py* que mantiene los subscriptores SSE en memoria por *user_id*.

Las dependencias se inyectan con el sistema de *Depends* de FastAPI: *get_db* abre una sesión SQLAlchemy, *get_current_user* valida el JWT del header Authorization y carga el User desde la base. Los settings se leen con *pydantic-settings* desde el archivo *.env*, lo que da validación de tipos y errores al arrancar si falta alguna variable.

7. Modelo de datos

Todo se modela con SQLAlchemy 2.0 usando la sintaxis declarativa moderna *Mapped/mapped_column*. IDs en UUID (no auto-increment) para evitar colisiones si en el futuro se migra a multi-tenant y se sincronizan bases. Estas son las tablas:

Tabla	Columnas y notas
users	id (UUID), store_name, email único e indexado, password (hash bcrypt), monthly_goal (Numeric 12,2)
categories	id, user_id (FK users), name, created_at.
products	id, user_id, name, price (Numeric 10,2), created_at. Relación many-to-many con categories via tabla
product_categories	Tabla puente con product_id y category_id como PK compuesta.
sales	id, user_id, invoice_number, payment_method ENUM(card,qr), card_type ENUM(credit,debit), card_b
sale_items	id, sale_id (FK), product_id (FK), quantity, subtotal. Carga eager con joined load.
product_stock	id, user_id, product_id, current_stock (int), updated_at. UNIQUE constraint sobre (user_id, product_id)
demand_predictions	id, user_id, product_id, predicted_quantity, period_start, period_end, confidence, recommended_puro

Todos los enums son ENUMs reales de Postgres (no strings sueltos), lo que da validación a nivel de motor de base. Las fechas son DateTime con timezone, siempre guardadas como UTC. Los montos usan Numeric con precisión y escala definidas para no perder centavos por floats.

8. Endpoints / API REST disponibles

Todos los endpoints (excepto register, login y stream) requieren JWT en el header *Authorization: Bearer <token>*:

Endpoint	Para qué sirve
POST /auth/register	Crea usuario nuevo.
POST /auth/login	Devuelve access_token JWT.
POST /auth/refresh	Renueva token aún con uno expirado (gracia).
GET /auth/me	Datos del usuario actual.
PUT /auth/me/goal	Setea o actualiza la meta mensual.

GET /dashboard/summary	KPIs, gráficos, top productos, top clientes, deltas vs período previo. Param: period=to
POST /sales/	Crear venta manual.
GET /sales/	Listar ventas con filtros y paginación.
GET /stream/sales	SSE con ventas nuevas en vivo (token por query string).
GET/POST /categories/	Listar y crear categorías.
GET/POST /products/	Listar y crear productos.
PUT /products/{id}/categories	Actualizar categorías de un producto.
GET /forecasting/recommendations	Lista priorizada de compras sugeridas.
GET /forecasting/stock	Stock actual por producto.
PUT /forecasting/stock/{product_id}	Setear stock de un producto.
POST /insights/chat	Chat con la IA del negocio.
GET /insights/briefing	Briefing diario.
GET /insights/alerts	Alertas detectadas automáticamente.
POST /clover/webhook	Recibe eventos push de Clover.
POST /clover/sync	Pull manual de las últimas órdenes.
GET /health	Liveness check del servicio.

9. Frontend en profundidad

El frontend es una SPA en React 18 servida por Vite 5. La estructura está en *frontend/src/* con tres carpetas centrales:

Carpeta	Qué hay adentro
pages/	Una pantalla por archivo: Login, Register, Dashboard, Insights, Products, Categories, PurchasePattern
components/	Componentes reutilizables: Sidebar, Header, ChartCard (wrapper de Recharts), StatCard (KPI card),
context/	AuthContext con AuthProvider, useAuth() y un helper apiFetch que adjunta el token y maneja el JSON

El ruteo se hace con *react-router-dom v6*. Hay un guard simple en *App.jsx*: si *useAuth().user* es null se muestra la pantalla de Login (y opcionalmente Register en */register*); si hay user, se renderiza el layout con Sidebar + Header y las páginas detrás. El modo oscuro se maneja con la clase *dark* de Tailwind aplicada al wrapper raíz, y se conserva en estado local.

Bibliotecas clave del frontend:

- **Recharts** para todos los gráficos (LineChart, BarChart, PieChart, ResponsiveContainer). Permite hacer responsive sin tocar canvas.
- **Tailwind CSS** con dark mode controlado por clase, customización en *tailwind.config.js*, plus algunas variables CSS (--scifi-bg, etc.) para acentos visuales.
- **Lucide React** para los íconos (DollarSign, ShoppingCart, Package, Cloud, RefreshCw, etc.).
- **Vite** como bundler/dev-server, con hot module replacement en menos de 200ms.

El componente *Dashboard.jsx* es el más grande: carga */dashboard/summary* según el período elegido, además de */insights/alerts*, */insights/briefing* y */auth/me* en paralelo con Promise.all. Tiene un botón *Sync Clover* que dispara */clover/sync* y, si trae ventas nuevas, refresca todo. La meta mensual

se edita inline desde el dashboard también, sin tener que ir a Settings.

10. Inteligencia Artificial conversacional

La parte de IA usa **Groq** como proveedor de inferencia (tiene una API gratuita con muy baja latencia) y el modelo **llama-3.3-70b-versatile**. La integración se hace con el SDK oficial de Groq desde el router *insights.py*.

Para cada interacción se arma un *system prompt* que **no es genérico**: incluye datos reales del negocio (revenue total, total de ventas, ticket promedio, métodos de pago, top productos, clientes únicos y tasa de retorno) calculados al momento desde la base. Esto convierte al modelo en un asesor que ya tiene contexto del negocio, no en un chatbot genérico.

Tres usos distintos:

- **Briefing**: max_tokens=200, estructura forzada por prompt en 2-3 oraciones (dato concreto + insight + acción).
- **Chat**: max_tokens=500, el historial de la conversación se pasa como mensajes encadenados para que la IA mantenga contexto entre repreguntas.
- **Si falta GROQ_API_KEY**: el briefing devuelve un mensaje educado pidiendo configurarla, en vez de explotar.

11. Machine Learning de forecasting

El servicio *app/services/ml_forecasting.py* es el módulo más técnico del backend. Pipeline completo:

Paso 1 — Agregación

Se hace una consulta SQL que agrupa *sale_items* por producto y semana ISO (con *date_trunc('week', sold_at)* de Postgres), sumando la cantidad vendida. El resultado se trae a pandas en un DataFrame.

Paso 2 — Feature engineering

Para cada producto se construyen estas features:

- **lag_1, lag_2, lag_3, lag_4**: cuántas unidades se vendieron 1, 2, 3 y 4 semanas atrás.
- **rolling_mean_4**: promedio móvil de las 4 últimas semanas (con shift para no filtrar la semana actual).
- **rolling_std_4**: desvío estándar de las 4 últimas semanas (rellenado con 0 si no hay suficientes datos).
- **week_of_year**: número de semana ISO del año.
- **month**: mes del año (1-12).
- **is_first_half**: si la semana cae en la primera mitad del mes (día ≤ 15).
- **product_enc**: label encoding del producto (cada producto único recibe un entero).

Paso 3 — Entrenamiento

Si hay al menos 4 semanas de datos (*MIN_WEEKS_FOR_ML*) y al menos 10 filas con features completas, se entrena un **XGBRegressor** con 100 estimadores, max_depth=4, learning_rate=0.1, random_state=42. Si no, se cae al fallback de *rolling average* de 4 semanas — esto evita el problema típico del cold-start donde un modelo se entrena con poca data y predice basura.

Paso 4 — Confianza

La confianza se computa como $1 - MAE / \max(y)$ sobre el set de entrenamiento, clipeada a [0.40, 0.99] para no mostrar nunca un número ni demasiado alto ni demasiado bajo en el frontend. Para el fallback promedio se hardcodea confianza 0.5.

Paso 5 — Predicción de la próxima semana

Para cada producto se arma una fila de features con los lags de las últimas 4 semanas observadas y el calendario de la próxima semana ISO. Se predice con el modelo, se clamping a no-negativos, y se redondea.

Paso 6 — Cálculo de recomendación y persistencia

$recommended_purchase = \max(0, predicted - current_stock)$. Se borran las predicciones viejas del usuario y se insertan las nuevas en la tabla *demand_predictions*. Se calcula un nivel de alerta (crítico / moderado / ok) según el ratio entre lo que hay que comprar y lo predicho. La lista se devuelve ordenada por urgencia descendente.

12. Integración con Clover POS

El servicio *app/services/clover_service.py* se comunica con la API REST de Clover (v3) usando **httpx**. Hoy las credenciales (*CLOVER_MERCHANT_ID* y *CLOVER_ACCESS_TOKEN*) vienen del archivo *.env*, lo que es funcional para un solo comercio pero incompatible con multi-tenant (próxima fase).

Funciones expuestas:

- **fetch_clover_order(order_id)**: trae una orden expandida (lineItems + payments).
- **fetch_clover_orders(limit)**: trae las últimas N órdenes ordenadas por createTime DESC.
- **_map_payment_info(order)**: detecta si el pago fue con tarjeta (y mapea VISA/MC/MASTERCARD/AMEX a nuestros enums) o QR.
- **_map_clover_order(order, user_id, db)**: convierte una orden Clover en un objeto Sale con su lista de SaleItems, creando productos nuevos si el nombre no existía aún para el usuario.
- **sync_clover_orders(user_id, db, limit)**: pull masivo idempotente — saltea órdenes ya importadas detectando duplicados por *clover_order_id* único.
- **process_clover_webhook(event_type, object_id, user_id, db)**: handler del webhook. Filtra eventos que no sean de tipo ORDER, evita duplicados y trae el detalle de la orden recién creada.

Conversión de unidades Clover: los precios vienen en centavos y la cantidad como *unitQty* en milésimas, por eso hay divisiones por 100 y agrupaciones por 100 al mapear.

13. Streaming en tiempo real

Server-Sent Events implementado con **sse-starlette**. Hay un *event_manager* que mantiene un dict *user_id* → *set de queues asyncio*. Cuando se crea una venta nueva, el router de */sales/* llama a *event_manager.broadcast(user_id, payload)*, que encola el evento para todos los suscriptores del usuario. La ruta *GET /stream/sales?token=...* valida el JWT por query string (los EventSource del browser no permiten headers custom), se suscribe al canal y streamea eventos *new_sale* hasta que el cliente cierra la conexión, momento en el cual se hace cleanup automático.

14. Autenticación y seguridad

El sistema usa JWT firmados con HS256 (un solo secreto compartido). El token contiene *sub=user.id* y un *exp* configurable (default 60 minutos). El hash de contraseñas es **bcrypt** vía *passlib*, lo que es el estándar industrial. Hay un endpoint */auth/refresh* que acepta tokens recién expirados (gracia configurable) para renovar sin obligar al login.

Notas de seguridad actuales (declaradas como deuda):

- CORS hoy está abierto con `allow_origins=["*"]` — hay que restringirlo a los dominios reales antes de producción.
- El webhook de Clover no verifica firma HMAC todavía — cualquiera con la URL podría inyectar ventas. En fase de hardening hay que validar la firma.
- `JWT_SECRET` tiene un default tipo *change-me-in-production* en el `.env` de ejemplo — fácil de olvidar al deployar.
- El stream usa el token por query string, lo que es funcional pero queda en logs si no se filtran.

15. Infraestructura local y deploy

El `docker-compose.yml` define tres servicios:

Servicio	Configuración
postgres	Postgres 15.3 Alpine. Volumen <code><i>pgdata</i></code> para persistencia. Carpeta <code><i>initdb</i></code> montada en <code>docker-e</code>
api	Build desde <code><i>backend</i></code> . Lee <code><i>backend/.env</i></code> . Expone 8000. Monta el código fuente como volumen
frontend	Build desde <code><i>frontend</i></code> . Expone 3000. Volumen del código + volumen anónimo para <code><i>node_modules</i></code>

Tras un `docker compose up --build -d` el sistema queda listo en `http://localhost:3000` (frontend) y `http://localhost:8000` (API). Las tablas se crean automáticamente al primer arranque del backend (DDL en lifespan, no Alembic). Para cargar la data demo se corre `docker compose exec -T api python seed_demo.py`.

16. App Android (Kotlin) — estado actual

La carpeta `CloverIAMarketing/` contiene un proyecto Android nativo con Gradle KTS. Tiene su propio `ARCHITECTURE.md` que define la estructura. **Estado actual:** implementadas la pantalla de login y un dashboard básico que consume la misma API REST del backend. **Lo que falta:** productos, categorías, ventas, insights, forecasting, settings, pull-to-refresh, modo offline con cache local, notificaciones push, ícono e identidad visual final. Es la primera fase del roadmap porque sin la app el comerciante no puede operar desde el celular en el mostrador.

17. Estado del proyecto, deuda técnica y próximos pasos

El MVP es **funcional y demostrable**: con la data demo se puede loguear, ver el dashboard completo, conversar con la IA, ver alertas, correr forecasting con XGBoost, hacer sync con Clover si hay credenciales, y recibir ventas en vivo por SSE. Lo que bloquea pasar a producción con clientes reales:

Issue	Por qué importa
■ P0 · Token Clover único en <code>.env</code>	Imposible servir a más de un comercio. Necesita OAuth v2 con tokens por usuario en
■ P0 · Webhook sin verificación HMAC	Cualquiera con la URL puede inyectar ventas falsas.
■ P0 · CORS abierto a *	Vector de CSRF y leak de tokens.
■ P0 · JWT_SECRET default	Riesgo si se olvida cambiar al deployar.
■ P1 · DDL al startup (sin Alembic)	Sin rollback de schema. Alembic ya está instalado pero no se usa.
■ P1 · Sin tests de integración	Cualquier cambio puede romper la sincronización silenciosamente.
■ P1 · Sin observabilidad	Ni logs centralizados ni Sentry ni métricas.

■ P1 · Sin reset password / verif. email	UX bloqueante para self-service.
■ P2 · Sin paginación en algunas listas	Performance se degrada con histórico grande.
■ P2 · Sin caché en dashboard	Recalcula KPIs en cada request.
■ P2 · Android incompleto	Web first es OK, perdemos terminales Clover Android por ahora.

18. Roadmap por fases

Resumen del roadmap definido en *ROADMAP.md* y *PLAN_ORQUESTADOR.md*:

Fase	Contenido
Fase 1 — Cerrar app Android	Productos, Categorías, Ventas, Insights, Forecasting, Settings, pull-to-refresh, offline básico
Fase 2 — Calidad y confiabilidad	Reset password, verificación de email, tests E2E, backups, logs centralizados, manejo de
Fase 3 — Sacar más jugo a Clover	OAuth v2, publicación en Clover App Market, sync de inventario bidireccional, sync de emp
Fase 3.5 — ML sobre data transaccional	Market Basket Analysis (Apriori/FP-Growth con mlxtend), segmentación RFM + KMeans, c
Fase 4 — Diferenciación con IA	IA proactiva, sugerencias de precios, detección de fraude, predicción de churn con model
Fase 5 — Crecimiento comercial	WhatsApp Business, programa de fidelidad, mini-e-commerce, marketplace entre comercia
Fase 6 — Escalar técnico	Multi-tenant real con business_id, roles y permisos, cache Redis para dashboard, cola Cel

19. Modelo de negocio sugerido

Esquema tentativo de monetización definido en *ROADMAP.md*:

Tier	Qué incluye
Free	Dashboard básico, hasta 100 ventas/mes.
Pro	\$29/mes. Insights IA, forecasting, alertas, sin límite de ventas.
Business	\$99/mes. Multi-local, multi-usuario, integraciones.
Enterprise	Precio a medida + soporte dedicado.

Distribución prevista: publicación en el Clover App Market una vez que el MVP esté endurecido y multi-tenant. Clover toma una comisión del 15-30% sobre el revenue del marketplace, lo que hay que considerar al fijar los precios.

Cierre · Atlas Nexus tiene hoy un MVP demostrable que combina un dashboard moderno en React, un backend FastAPI prolijo, ML real con XGBoost, IA conversacional con Llama 3.3, e integración con Clover por REST y webhooks. La diferenciación clara es **IA accionable** sobre data transaccional, no un dashboard más. Lo que sigue es endurecer el código y abrir el camino al Clover App Market.